

MID SEMESTER EXAMINATION — SOLUTIONS & MARKING SCHEME

Subject: Data Structure

Time: 1.5 Hours

Semester / Programme: 3rd Semester

Full Marks: 20

Instructions: Answer Question No. 1 (Compulsory) and any THREE from Question No. 2 to 5. Figures in brackets indicate marks.

SECTION - A (Compulsory)

1(a) How many different Binary Search Trees (BST) can be constructed with 4 distinct keys? [1]

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \prod_{i=2}^n \frac{n+i}{i}$$

Solution & Key Concept:

- Use the Catalan number formula for $n = 4$:
- $C_4 = (1/4+1) C\{2(4)\}4 = (1/5) C\{8\}4$
- Calculate $C\{8\}4 = (8 \times 7 \times 6 \times 5 / 4 \times 3 \times 2 \times 1) = 70$
- $C_4 = (70/5) = 14$

Answer: 14

Marking Rubric: [1 Mark] for correct final answer or valid formula substitution.

1(b) State the worst-case time complexity of searching an element in a Binary Search Tree (BST) containing n nodes, and specify the condition under which it occurs. [1]

$$T(n) = O(n)$$

Solution & Key Concept:

- Identify the worst-case scenario where a BST degenerates into a skewed tree (either completely left-skewed or right-skewed).
- In a skewed tree, every node has at most one child, resembling a linked list.
- Searching an element in such a tree requires traversing all n nodes in the worst case.

Answer: $O(n)$, occurs when the BST is completely skewed.

Marking Rubric: [1 Mark] for stating $O(n)$ and mentioning skewed tree structure.

1(c) Evaluate the postfix expression: 5 \; 3 \; + \; 8 \; 2 \; - \; \; *

[1]

Solution & Key Concept:

- Scan tokens from left to right: push 5, push 3 onto stack.
- Encounter '+': pop 3 and 5, compute $5 + 3 = 8$, push 8.
- Push 8, push 2 onto stack.
- Encounter '-': pop 2 and 8, compute $8 - 2 = 6$, push 6.
- Encounter '*': pop 6 and 8, compute $8 \times 6 = 48$, push 48.

Answer: 48

Marking Rubric: [1 Mark] for correct final evaluated result.

1(d) What is the primary advantage of a Circular Queue over a Linear Queue implemented using a static array? [1]

Solution & Key Concept:

- In a linear static array-based queue, once the rear pointer reaches the maximum capacity, no more elements can be inserted even if spaces are available at the front due to previous deletions ('false overflow').
- A circular queue connects the end of the array back to the front, efficiently utilizing empty spaces created by dequeued elements without requiring costly shifting operations.

Answer: Efficient space utilization and avoidance of false overflow condition.

Marking Rubric: [1 Mark] for mentioning avoidance of false overflow / memory reuse.

1(e) Define a sparse matrix and write its compressed triple form (Row, Column, Value) storage structure representation parameters for a matrix of size $M \times N$ with *non-zero* elements K . [1]

Storage = $(K + 1) \times 3$

Solution & Key Concept:

- A sparse matrix is a matrix in which the majority of the elements are zero.
- The triple form stores non-zero elements in a 2D array of size $(K+1) \times 3$.
- The first row stores total rows M , total columns N , and total non-zero count K . Subsequent rows store the row index, column index, and value for each non-zero element.

Answer: A 2D array of size $(K+1) \times 3$ storing row, col, and value.

Marking Rubric: [0.5 Mark] for definition, [0.5 Mark] for $(K+1) \times 3$ triple structure explanation.

SECTION - B (Answer any THREE)

Answer any *THREE* questions from Q.2 to Q.5. Each question carries 5 marks.

Q.2. (a) Write an efficient algorithm/C function to reverse a singly linked list iteratively. Explain its time and space complexity. [3]

Solution for (a):

- Initialize three pointers: `prev = NULL`, `current = head`, `next = NULL`.
- Traverse the list: while `current` is not `NULL`, do: a. `next = current->next` b. `current->next = prev` c. `prev = current` d. `current = next`
- After loop finishes, set `head = prev`.
- Time Complexity: $O(n)$ as we traverse the list of n nodes once. Space Complexity: $O(1)$ as only pointers are used.

Result / Conclusion: Iterative reversal algorithm using 3 pointers with $O(n)$ time and $O(1)$ space.

Marking Rubric: [2 Marks] for correct algorithm/code logic, [1 Mark] for complexity analysis.

(b) Explain how two sparse matrices can be added efficiently using compressed coordinate (triple) representation. Give the structural outline of the algorithm. [2]

Solution for (b):

- Check if dimensions $(M \times N)$ of both sparse matrices match. If not, addition is impossible.
- Compare row and column indices of non-zero elements from both representations simultaneously using two pointers (i and j).
- If row/col indices match, add their values. If the sum is non-zero, store it in the result matrix.
- If indices do not match, copy the element with the smaller index (row-major order) into the result matrix and increment that pointer.

Result / Conclusion: Parallel pointer traversal algorithm comparing (row, col) tuples yielding $O(K_1 + K_2)$ complexity.

Marking Rubric: [1 Mark] for dimension check and pointer comparison logic, [1 Mark] for merging/addition rules.

Q.3. (a) Convert the following infix expression into its equivalent postfix expression using a stack showing the step-by-step trace: $Q = (A + B) * C - (D - E) / (F + G)$

[3]

Solution for (a):

- Token '(': push '('
- Token 'A': output A
- Token '+': push '+'
- Token 'B': output B
- Token ')': pop until '(', output AB+
- Token '*': push '*'
- Token 'C': output C → AB+C*
- Token '-': precedence check, push '-'
- Token '(': push '('
- Token 'D': output D
- Token '-': push '-'
- Token 'E': output E → DE-
- Token ')': pop until '(', output DE-
- Token '/': push '/'
- Token '(': push '('
- Token 'F': output F
- Token '+': push '+'
- Token 'G': output G → FG+
- Token ')': pop until '(', output FG+ / DE- / ...
- Final Postfix: AB+C*DE-FG+/-

Result / Conclusion: AB+C*DE-FG+/-

Marking Rubric: [2 Marks] for intermediate trace/stack operations, [1 Mark] for correct final postfix expression.

(b) Write a recursive function in C/C++ to find the height (or maximum depth) of a binary tree.

[2]

$Height(T) = 1 + \max(Height(T_{left}), Height(T_{right}))$

Solution for (b):

- Define base case: if the node pointer is `NULL`, return `0`.
- Recursively compute height of left subtree: `int leftHeight = height(node->left);`
- Recursively compute height of right subtree: `int rightHeight = height(node->right);`
- Return `1 + max(leftHeight, rightHeight)`.

Result / Conclusion: Recursive function returning `1 + max(left, right)` with base case `NULL → 0`.

Marking Rubric: [1 Mark] for base case condition, [1 Mark] for recursive relation and return statement.

- Q.4. (a)** Construct a Binary Search Tree (BST) by inserting the following sequence of elements one by one: 50, 30, 70, 20, 40, 60, 80. Then, write the post-order traversal sequence of the resulting BST. [3]

Solution for (a):

- Insert 50 (Root)
- Insert 30 (Left child of 50)
- Insert 70 (Right child of 50)
- Insert 20 (Left child of 30)
- Insert 40 (Right child of 30)
- Insert 60 (Left child of 70)
- Insert 80 (Right child of 70)
- Post-order traversal (Left, Right, Root): - Subtree rooted at 30: Left(20), Right(40), Root(30) -> 20, 40, 30 - Subtree rooted at 70: Left(60), Right(80), Root(70) -> 60, 80, 70 - Root 50: -> 20, 40, 30, 60, 80, 70, 50

Result / Conclusion: Post-order: 20, 40, 30, 60, 80, 70, 50

Marking Rubric: [1.5 Marks] for correct BST structure, [1.5 Marks] for accurate post-order traversal.

- (b)** Write a pseudo-code or C function to traverse a binary tree in ****in-order**** using an explicit user-defined stack (non-recursive). [2]

Solution for (b):

- Initialize `curr = root` and create an empty stack.
- Run a loop while `curr != NULL` or stack is not empty:
- a. While `curr != NULL`, push `curr` onto stack and move `curr = curr->left`.
- b. Pop the top node from stack, print/visit its data (`curr = stack.pop()`).
- c. Shift to its right subtree: `curr = curr->right`.

Result / Conclusion: Non-recursive in-order traversal algorithm using stack.

Marking Rubric: [1 Mark] for correct stack push-left logic, [1 Mark] for pop and right-shift logic.

- Q.5. (a)** Construct a unique Binary Tree from the given traversal sequences: - Preorder Traversal : `A B D E C F`
- In-order Traversal : `D B E A F C` [3]

Solution for (a):

- Identify root from Preorder: First element is `A`. So `A` is the root.
- Locate `A` in In-order: Elements to the left (`D B E`) form the left subtree, elements to the right (`F C`) form the right subtree.
- For Left Subtree (`D B E`), Preorder gives root `B`. In In-order (`D` left, `E` right), `D` is left child of `B`, `E` is right child of `B`.
- For Right Subtree (`F C`), Preorder gives root `C`. In In-order (`F` left), `F` is left child of `C`.
- Assemble the complete binary tree structure.

Result / Conclusion: Root: A, Left child of A: B (with children D and E), Right child of A: C (with left child F).

Marking Rubric: [1.5 Marks] for correct identification of subtrees, [1.5 Marks] for final tree reconstruction.

- (b)** Write a complete C/C++ function to insert a new node at the beginning of a Doubly Linked List (DLL), updating both pointers correctly. [2]

Solution for (b):

- Allocate memory for the new node: `Node\* newNode = new Node(); newNode->data = val;`
- Set `newNode->next = (\*head\_ref);`
- Set `newNode->prev = NULL;`
- If the original list is not empty (`\*head\_ref != NULL`), set `(\*head\_ref)->prev = newNode;`
- Update the head pointer: `\*head\_ref = newNode;`

Result / Conclusion: Function implementation for DLL front insertion with proper `prev` and `next` pointer adjustments.

Marking Rubric: [1 Mark] for node creation and pointer linking, [1 Mark] for handling head pointer and edge cases.

--- END OF PAPER ---